

Mongo DB

Complete Reference

What you'll master:

Documents · Collections · BSON

SQL vs MongoDB mapping

Database & Collection commands

Create — insertOne · insertMany

Read — find · findOne · filters

Update — updateOne · replaceOne

Delete — deleteOne · deleteMany

What is MongoDB?

A NoSQL document database — stores data as flexible JSON-like documents

MongoDB is...

An open-source NoSQL database that stores data as JSON-like documents (called BSON internally). Unlike SQL databases that use rigid tables with rows and columns, MongoDB lets each document have its own structure — perfect for modern, fast-changing apps.



Document-based

JSON-like flexible storage



Schema-less

Different fields per doc — no fixed structure



Scalable

Easily handles millions across many servers



Fast

Optimised for high-volume read/write



Open-source

Free to use, Atlas cloud version available



Cross-platform

Runs on Windows, Mac, Linux

The MongoDB Hierarchy

Database → Collection → Document → Field

Database

The container — like an entire office building

```
company
```



Collection

Group of similar documents — like a floor

```
employees
```



Document

One record — like a single paper file

```
{ _id, name, age, city }
```



Field

One property — like one line on that paper

```
name: 'Aarav Sharma'
```

SQL vs MongoDB

Map the terminology you already know to MongoDB

SQL (Relational)	MongoDB (NoSQL)	Meaning
Database	Database	Same — the top container
Table	Collection	Group of records
Row	Document	ONE record
Column	Field	One property/attribute
Primary Key	<code>_id</code>	Unique identifier
JOIN	Embedded / \$lookup	Linking related data
SELECT *	<code>db.users.find()</code>	Get all records

Database Commands

Creating · switching · listing · dropping databases

SYNTAX

```
show dbs
```

Lists all databases on the MongoDB server with their sizes.

```
show dbs
```

SYNTAX

```
use <name>
```

Creates new DB if it doesn't exist, OR switches to existing one.

```
use company
```

SYNTAX

db

Refers to the CURRENT database — used as 'db.collection.command()'.

```
db → outputs: company
```

SYNTAX

```
db.dropDatabase()
```

Permanently deletes the entire current database. No undo!

```
use company; db.dropDatabase()
```

Collection Commands

Creating · listing · dropping collections

SYNTAX

```
db.createCollection('<name>')
```

Creates a new collection inside the current database. Returns { ok: 1 } on success.

```
db.createCollection('employees')
```

SYNTAX

```
show collections
```

Lists all collections in the current database.

```
show collections
```

SYNTAX

```
db.<col>.drop()
```

Permanently deletes a collection AND all its documents.

```
db.employees.drop()
```

Important

drop() and dropDatabase() are PERMANENT — there is no undo. In production, always backup first.

Create — Inserting Documents

insertOne · insertMany

SYNTAX

```
db.<col>.insertOne({ ...document })
```

Inserts a SINGLE document. MongoDB auto-generates the `_id` field.

```
db.employees.insertOne({ name: 'Aarav', age: 28, city: 'Mumbai' })
```

SYNTAX

```
db.<col>.insertMany([ {...}, {...}, ... ])
```

Inserts MULTIPLE documents at once. Takes an ARRAY (square brackets) of objects.

```
db.employees.insertMany([ {name:'A',age:25}, {name:'B',age:30} ])
```

Key difference

insertOne takes ONE object {...} · insertMany takes an ARRAY of objects [...]

Read — Fetching Documents

find · findOne

SYNTAX

`db.<col>.find()` / `db.<col>.find({ filter })`

Returns ALL matching documents. With no filter — returns everything.

```
db.employees.find({ city: 'Mumbai' })
```

SYNTAX

`db.<col>.findOne({ filter })`

Returns ONLY the FIRST matching document (or null if nothing matches).

```
db.employees.findOne({ role: 'Manager' })
```

find()

Returns ALL matches as an array

findOne()

Returns ONLY the first matching doc

Read — Comparison Operators

\$gt · \$lt · \$gte · \$lte · \$eq · \$ne

\$gt

Greater than (>)

```
{ age: { $gt: 30 } }
```

\$lt

Less than (<)

```
{ age: { $lt: 30 } }
```

\$gte

Greater than or equal

```
{ age: { $gte: 30 } }
```

\$lte

Less than or equal

```
{ age: { $lte: 30 } }
```

\$eq

Equal (=)

```
{ age: { $eq: 30 } }
```

\$ne

Not equal (!=)

```
{ age: { $ne: 30 } }
```

 **Combine for ranges**

`db.employees.find({ salary: { $gte: 50000, $lte: 80000 } })` → salary between 50K and 80K

Read — Projection

Show only specific fields — not the whole document

SYNTAX

```
db.<col>.find( filter, { field1: 1, field2: 1, _id: 0 } )
```

Second argument is the projection. Use 1 to INCLUDE a field, 0 to EXCLUDE.

```
db.employees.find({}, { name: 1, city: 1, _id: 0 })
```

Without projection:

```
db.employees.find()  
  
// Returns EVERYTHING:  
{  
  _id: ObjectId('...'),  
  name: 'Aarav',  
  age: 28,  
  city: 'Mumbai',  
  role: 'Developer',  
  salary: 65000,  
  department: 'Tech'  
}
```

With projection:

```
db.employees.find(  
  {},  
  { name: 1, city: 1, _id: 0 }  
)  
  
// Returns ONLY:  
{  
  name: 'Aarav',  
  city: 'Mumbai'  
}
```

Update — Modifying Documents

updateOne · updateMany · replaceOne

SYNTAX

```
db.<col>.updateOne( filter, { $set: { field: value } } )
```

Updates the FIRST matching document. Only changes specified fields (\$set).

```
db.employees.updateOne({ name: 'A' }, { $set: { salary: 75000 } })
```

SYNTAX

```
db.<col>.updateMany( filter, { $set: { field: value } } )
```

Same as updateOne — but updates ALL matching documents.

```
db.employees.updateMany({ role: 'Dev' }, { $set: { dept: 'Tech' } })
```

SYNTAX

```
db.<col>.replaceOne( filter, { ...entireNewDocument } )
```

REPLACES the whole document — fields NOT in the new doc are DELETED!

```
db.employees.replaceOne({ name: 'A' }, { name: 'A', age: 30 })
```

⚠ updateOne = changes specific fields · replaceOne = wipes the WHOLE doc and replaces it

Delete — Removing Documents

deleteOne · deleteMany · drop · dropDatabase

SYNTAX

```
db.<col>.deleteOne({ filter })
```

Deletes ONLY the first document matching the filter.

```
db.employees.deleteOne({ name: 'Tarun' })
```

SYNTAX

```
db.<col>.deleteMany({ filter })
```

Deletes ALL documents matching the filter.

```
db.employees.deleteMany({ role: 'Intern' })
```

SYNTAX

```
db.<col>.drop()
```

Drops the entire collection — every document GONE.

```
db.employees.drop()
```

SYNTAX

```
db.dropDatabase()
```

Drops the ENTIRE database — all collections gone.

```
db.dropDatabase()
```

 **Permanent — no undo!**

deleteMany({}) deletes ALL documents · drop() removes the collection · dropDatabase() destroys everything. Always double-check.

Bulk Write — What & Why

Multiple write operations in ONE command

What is Bulk Write?

Run MULTIPLE write operations (insertOne, updateOne, deleteOne, etc.) in a SINGLE command — one round-trip to the database instead of many. Faster, cleaner, and supports atomicity options.

Supported operations inside bulkWrite:

insertOne

Insert one document

updateOne

Update first match

updateMany

Update all matches

replaceOne

Replace whole doc

deleteOne

Delete first match

deleteMany

Delete all matches

Ordered vs Unordered

Two modes — and what happens when something fails

ORDERED (default)

- Operations run one by one
- If ONE fails → ALL stop
- Slower but safer
- Best when order matters

```
db.users.bulkWrite([
  { insertOne: {...} }, // ✅
  { updateOne: {...} }, // ❌ fails
  { deleteOne: {...} }, // ⚪ skipped
  { insertOne: {...} }, // ⚪ skipped
])
```

UNORDERED

- Operations run in parallel
- Failure doesn't stop others
- Faster but less safe
- Best for independent ops

```
db.users.bulkWrite([
  { insertOne: {...} }, // ✅
  { updateOne: {...} }, // ❌ fails
  { deleteOne: {...} }, // ✅ runs
  { insertOne: {...} }, // ✅ runs
], { ordered: false })
```

🔑 Ordered is the default — explicitly set `{ ordered: false }` to enable unordered mode

Bulk Write — Complete Example

Three operations in one command

SYNTAX

```
db.<col>.bulkWrite([ {op1}, {op2}, ... ], { ordered: T/F })
```

Each op is an object with the operation name as key. Each op type needs specific keys (document, filter, update, etc).

```
db.employees.bulkWrite([ {insertOne:{...}}, {updateOne:{...}} ])
```

```
db.employees.bulkWrite([  
  // ❶ INSERT a new employee  
  { insertOne: { document: { name: 'Aryan', age: 27, role: 'Developer' } } },  
  
  // ❷ UPDATE Aarav's salary  
  { updateOne: { filter: { name: 'Aarav' }, update: { $set: { salary: 80000 } } } },  
  
  // ❸ DELETE intern Tarun  
  { deleteOne: { filter: { name: 'Tarun' } } }  
])  
  
// Response: { insertedCount: 1, modifiedCount: 1, deletedCount: 1 }
```

Upsert — Update + Insert

Smart update that creates the doc if it doesn't exist

UPSERT = UPDATE + INSERT

If filter MATCHES → update that doc. · If filter has NO MATCH → create a NEW doc with the filter + update values.

SYNTAX

```
db.<col>.updateOne( filter, update, { upsert: true } )
```

Add upsert: true as the third option to updateOne. The new doc combines filter fields + the \$set fields.

```
db.employees.updateOne({ name: 'Rohit' }, { $set: { age: 30 } }, { upsert: true })
```

If Rohit EXISTS:

```
// Original doc:
{ name: 'Rohit', age: 25 }

// After upsert:
{ name: 'Rohit', age: 30 }

// upsertedCount: 0
// modifiedCount: 1
```

If Rohit DOESN'T exist:

```
// No matching doc...

// MongoDB creates:
{ _id: ..., name: 'Rohit', age: 30 }

// upsertedCount: 1
// upsertedId: ObjectId(...)
```